Lab 4: Block World

Scott Schmidl
sschmidl3@gatech.edu

1 HOW DOES YOUR AGENT WORK?

My Block World agent treats each problem as a planning task in a state space, with Means-End Analysis, Generate & Test, and Problem Reduction as guides. It starts by scanning the goal arrangement to identify, for each block, what it should sit on, i.e. another block or the table. From here, it builds a simple representation of final positions that finds what the correct support is for every block and whether it's currently in that position. Using this representation, the agent works through the goal stacks from bottom to top, focusing first on blocks that should be on the table and then on the blocks that should sit on top of them. This can be thought of as hierarchical planning, i.e. building the base structure first, then adding dependent pieces on top and is similar to partial-order planning that adheres to preconditions such as "the supporting block must be correctly placed before its dependent block can be placed".

The agent is, essentially, a pair of helper functions that implement the strategy. The `final_position` method checks whether a given block and all blocks under it already match their goal, which lets the agent avoid disturbing parts of the structure that are complete. The `clear_block_above` method performs the Means-End Analysis part on the chosen block. It checks if the block has something on top of it, and if it shouldn't, recursively removes the blocks above it until the block is clear. When removing blocks, the agent first tries to place them directly into their goal position, otherwise, it temporarily moves them to the table. Combined with `move_block`, which updates the representations and records the 2-tuple moves, this gives a Generate & Test loop, i.e. generate moves that clear and place blocks, test whether blocks match their goal positions, and only move on when conditions are satisfied.

2 HOW WELL DOES YOUR AGENT PERFORM?

Overall, the agent performs well on each task. My agent scores 36 out of 40 points by successfully completing each task and 16 optimally. The four cases that aren't optimal are arrangements 13, 17, 18, and 19, because they take more moves than allowed. For example, on arrangement 13, the agent uses 12 moves, while 10

moves is optimal; on arrangement 17, it uses 16 moves while 14 is optimal; on arrangement 18, it uses 22 moves while 20 is optimal; and on arrangement 19, it uses 25 moves while 23 is optimal. This suggests the agent is good at solving them but needs more work to find all shortest paths.

Given that the agent systematically works from goal bottoms to tops and uses the same clearing and placement logic for every block, it doesn't get stuck in cycles or illegal moves, and respects constraints such as "only top blocks can be moved". The main performance struggle shows up in problems with many interlocking goal stacks where several blocks can be temporarily placed in multiple reasonable spots. In those cases, the agent sometimes takes extra steps, i.e. moving a block to the table when it doesn't need to. Still, the fact that it solves every problem correctly indicates that its planning strategy is working, even if not fully optimized in all cases.

3 HOW EFFICIENT IS YOUR AGENT?

From a runtime perspective, my agent runs in time that is roughly proportional to the number of blocks and stacks in the problem. Each time it needs to clear or place a block, it scans the current arrangement to find the block, its support, and any block above it. In a worst case scenario this is linear in the number of blocks, and these scans happen a constant number of times per block. This gives a worst case time complexity on the order of $O(n^2)$, where n is the number of blocks, because for each of the n blocks the agent may have to look through $O(n)$ stacks and update `block_above` and `block_below`. Given that at most we see 26 blocks, the agent is still fast and the agent solves the problem quickly.

As the number of blocks rises, the agent's performance will scale predictably with the $O(n^2)$ pattern. Doubling the number of blocks roughly multiplies the amount of work, because there are more blocks to clear and more positions to check each time. The agent doesn't perform any exponential search of the full space. It's a deterministic goal-oriented procedure. This means that while it is not guaranteed to find the absolute shortest path for every problem, it avoids runtime explosions and remains manageable even when arrangements involve many stacks and blocks.

4 DOES YOUR AGENT DO ANYTHING CLEVER?

The agent's main clever idea is how it uses the goal configuration to guide every clearing and placement decision, instead of treating clearing as random or table-based. When it is forced to remove a block from on top of another block, it does not automatically put that block on the table. Instead, it first checks whether its goal block is already placed and clear. If so, it moves the block directly to that goal block, which helps reduce future work. This is a small but important use of its goal-oriented reasoning. It needs to treat each clearing move as an opportunity to make progress toward the goal arrangement, and not just as a way to free a block below it.

Another piece of cleverness is the way the agent preserves currently correct stacks. The `final_position` method walks down from a block to the table, ensuring each support matches the goal stack. If the whole chain matches, the agent commits to not disturbing it later. This acts like a constraint that prunes the agent's search space, because it essentially says "once the goal stack is built, never break it apart again". While the agent doesn't implement complex meta-reasoning, these checks make its behavior more human-like and less wasteful than a naive strategy that keeps tearing down and rebuilding stacks.

5 HOW DOES YOUR AGENT COMPARE TO A HUMAN?

My agent solves Block World in a way that is similar to how I would solve it. When I look at a problem, I first try to see which blocks need to become the bottoms of the final stacks, then I move blocks to the table until I can start building those goal stacks from the bottom up. In arrangement 8, for example, I would first move F to the table because it has to be on the bottom of a goal stack. Then, I would move C to the table so B becomes free, and since B belongs on the table, that gets moved next. Now F and B are in their places, so next E gets moved to B, D to F, C to D, I to C, H to E, G to I, and A to G. This all started, though, with getting F and B to the table. My solution for arrangement 8 is the optimal 10 move solution. This is the same idea my agent uses, i.e. clear blocks then place blocks as soon as their support is ready.

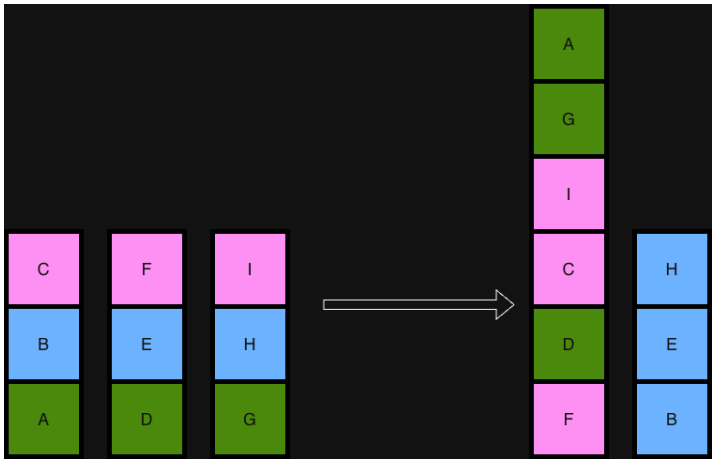


Figure 1—Arrangement 8: (F, Table) (C, Table) (B, Table) (E, B) (D, F) (C, D) (I, C) (H, E) (G, I) (A, G)

The main difference is that a human can be more flexible about their overall plan, while my agent is more rigid. I would describe my strategy as "move blocks to the table until I find one that belongs on the table or can go directly onto its support". That matches my agent, but a person can sometimes look farther ahead and notice a shorter sequence before making a move. My agent is better at remembering constraints and not making illegal moves, but a human may be better at spotting when a temporary move now will save two moves later. So overall, the agent is similar to how I reason through the stacks, but it's less flexible than a person when there are several good options.